UNIVERSIDAD PRIVADA DEL VALLE

FACULTAD: FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA: INGENIERÍA DE SISTEMAS INFORMÁTICOS

MATERIA: TECNOLOGIAS EMERGENTES 1

PRACTICA 16:

LI16 - Practica con Apache Hive

MATERIA: Tecnologías Emergentes 1

TEMA: LI16 - Practica con Apache Hive

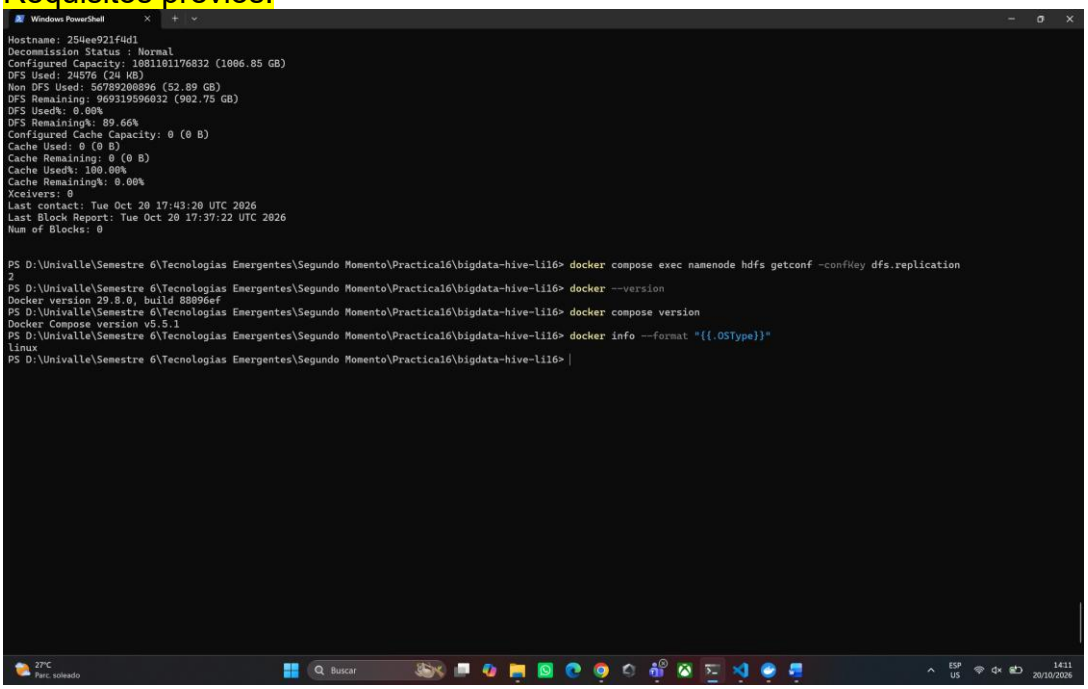
ESTUDIANTE: Alejandro Vidal

FECHA PRESENTACIÓN INFORME: 20/10/2026

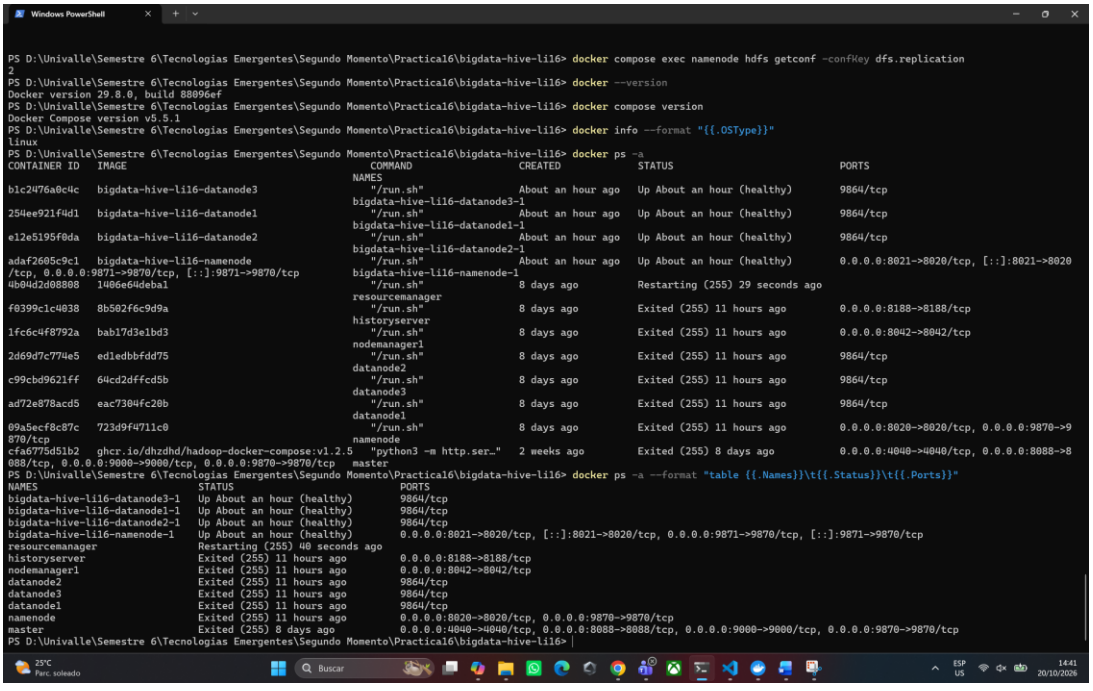
LI16 - Práctica con Apache Hive (1)

Para este laboratorio, usted debe contar con docker y docker-composer desde el sistema operativo Ubuntu de preferencia:

Requisitos previos:



1. (Opcional): Tomar en cuenta qué contenedores existen en su equipo; preferentemente, **debe estar vacío** para facilidad de interpretación.

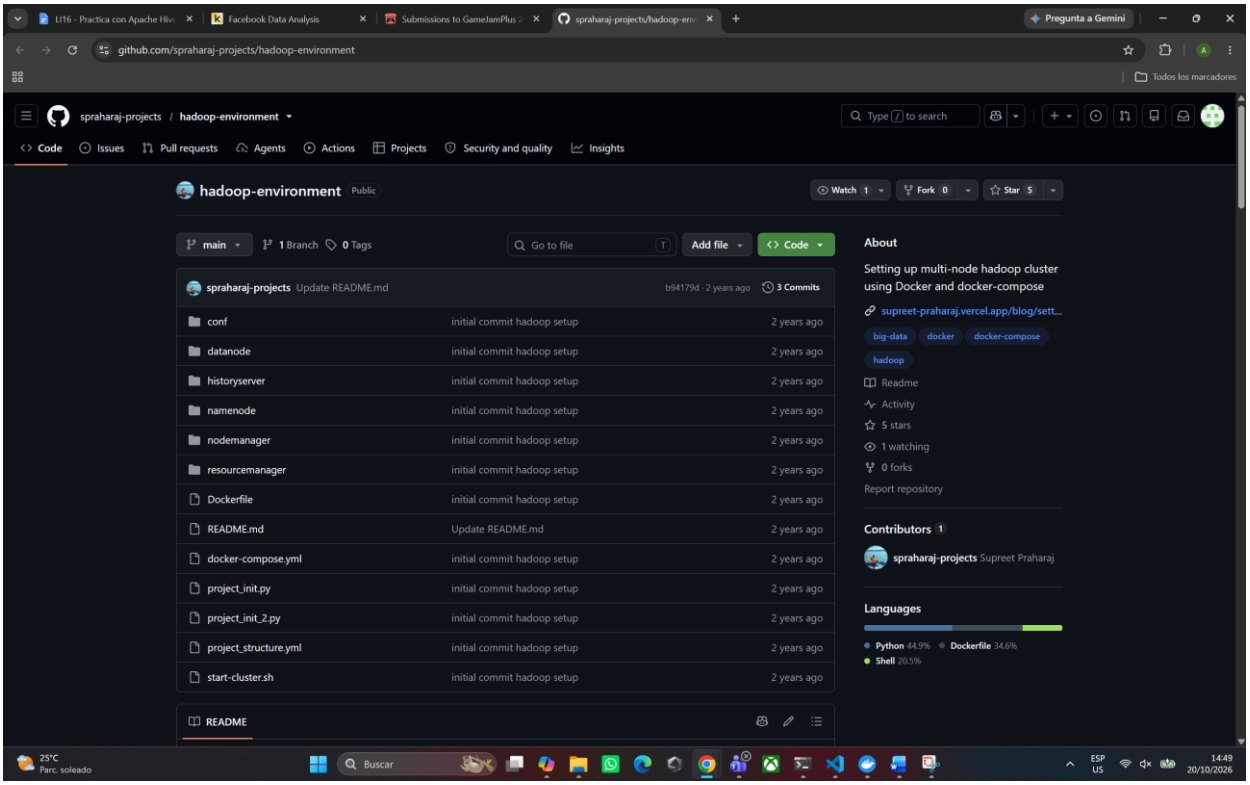


Se ejecutó el comando `docker ps -a` para identificar los contenedores existentes en Docker. Se observaron contenedores de la práctica anterior de Hadoop, en su mayoría detenidos,

además de cuatro contenedores activos correspondientes a la práctica LI16: un NameNode y tres DataNodes. También se identificó un contenedor antiguo ResourceManager en estado de reinicio. Los contenedores de LI16 utilizan nombres y recursos independientes para evitar conflictos con la infraestructura anterior.

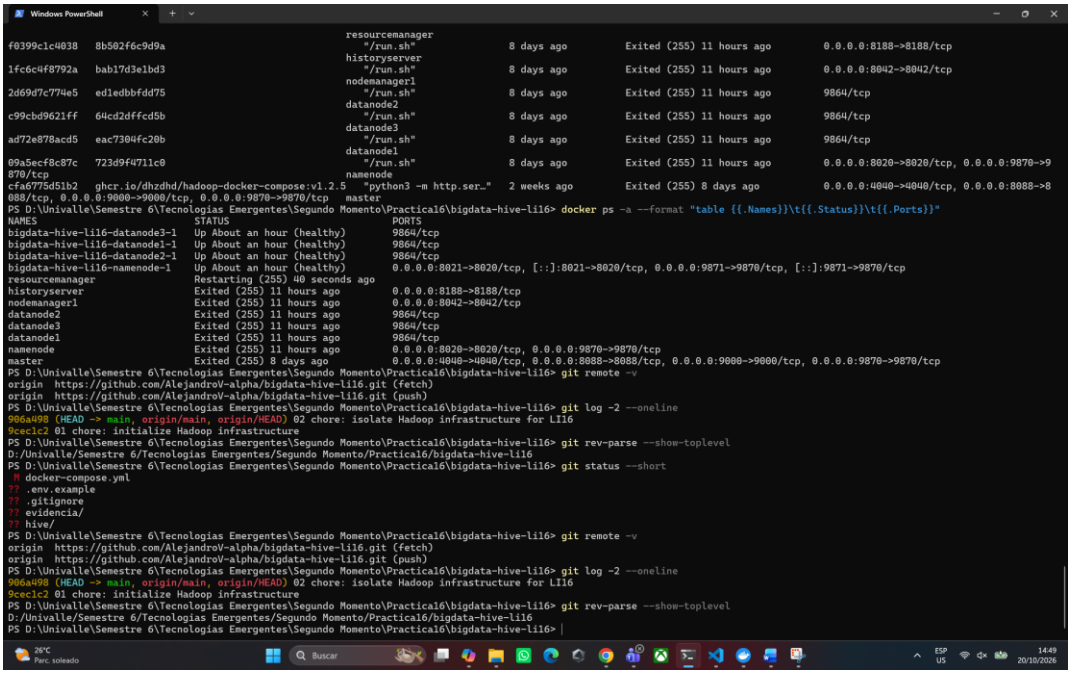
2. Deberá usar un ***repositorio diferente al de big data europe***, puede usar el que vio con su grupo.

Para desarrollar esta práctica se tomó como referencia el repositorio original [spraharaj-projects/hadoop-environment](), que proporciona una infraestructura de Apache Hadoop mediante Docker.

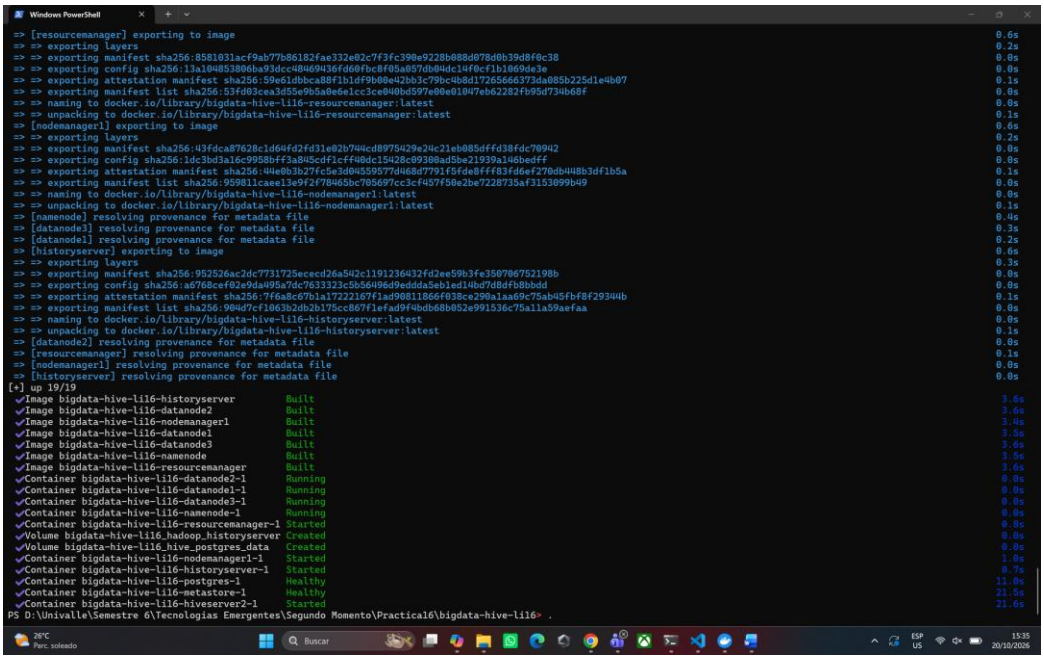


En una práctica anterior, nuestro grupo adaptó dicha infraestructura en el repositorio [bigdata-hdfs-grupo4](), incorporando un factor de replicación de 2 y actualizando configuraciones para mejorar la compatibilidad con versiones modernas de Docker.

Para la práctica LI16 se creó y clonó un nuevo repositorio denominado [bigdata-hive-li16](), reutilizando la infraestructura Hadoop corregida y preparando la integración de Apache Hive sin modificar el trabajo anterior.

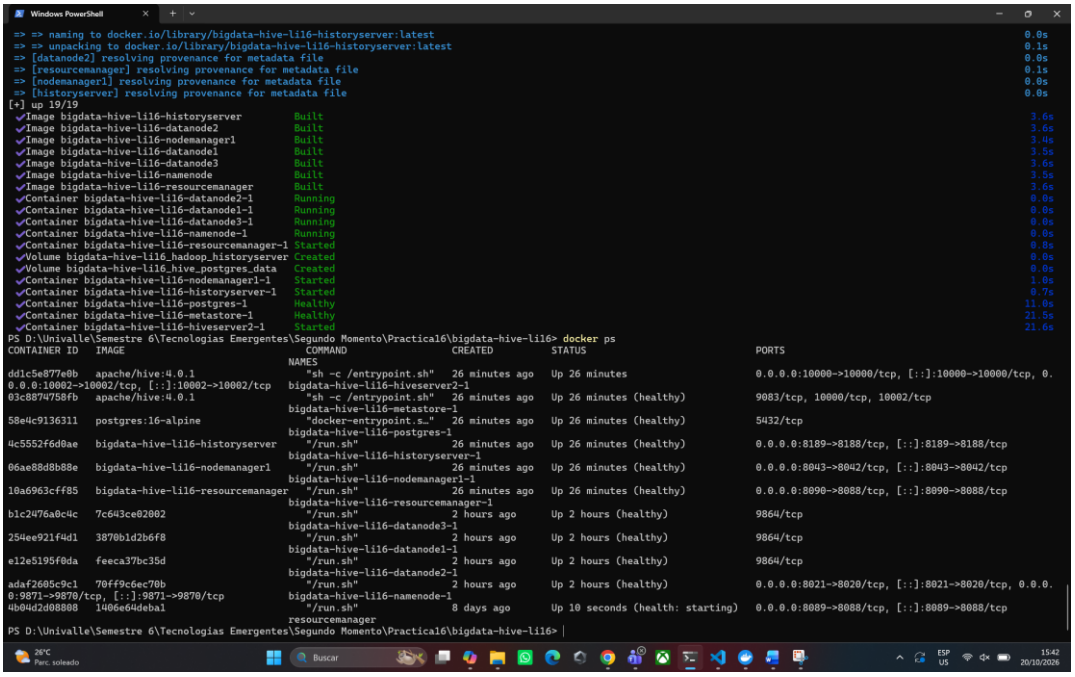


3. Debe ingresar a la carpeta que se descarga denominada “docker-hive” y ejecutar el comando: **docker-compose up -d,** segun la arquitectura de su repositorio



Se ejecutó el comando docker compose up -d --build desde el repositorio bigdata-hive-li16 para construir e iniciar los servicios del entorno. Docker Compose completó el despliegue de la infraestructura Hadoop y de los nuevos componentes de Apache Hive, incluyendo PostgreSQL, Hive Metastore y HiveServer2. Se utilizó Docker Compose V2, manteniendo una configuración independiente de la práctica anterior.

4. Listar los contenedores con el comando: docker ps



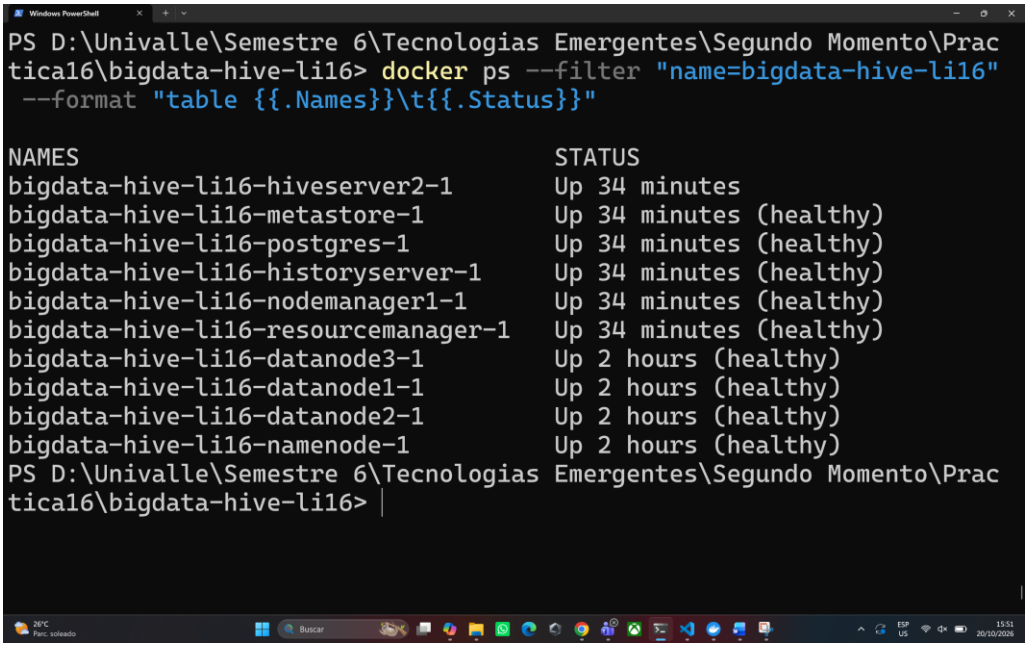
Se ejecutó el comando docker ps para verificar los contenedores en funcionamiento. Se identificaron los diez contenedores correspondientes a la práctica LI16, incluyendo los servicios de Hadoop HDFS, YARN, PostgreSQL y Apache Hive. Los contenedores se encontraban en ejecución y los servicios con comprobaciones de salud mostraron un estado saludable. También se observó un contenedor ResourceManager perteneciente a una práctica anterior.

5. De la lista anterior, realice un análisis reflexivo y describa en un párrafo cada uno de los contenedores haciendo un poco de investigación:

Contenedor	Relación y comprensión con la exposición
bigdata-hive-li16-hiveserver2-1	Este contenedor ejecuta HiveServer2, que permite conectarse a Apache Hive y realizar consultas mediante SQL. Su función es recibir las consultas de los usuarios, procesarlas y coordinar su ejecución con los demás servicios. En nuestra práctica será el componente que utilizaremos para crear la base de datos, definir la tabla de Facebook y consultar los registros almacenados en HDFS.
bigdata-hive-li16-metastore-1	Hive Metastore se encarga de administrar la información sobre las bases de datos y tablas creadas en Hive, incluyendo sus columnas, tipos de datos y ubicaciones de almacenamiento. Comprendo que este servicio es importante porque permite a Hive conocer la estructura de los datos sin tener que almacenarlos directamente. En nuestra práctica trabajará junto con PostgreSQL y HDFS.
bigdata-hive-li16-postgres-1	Este contenedor utiliza PostgreSQL como base de datos para guardar los metadatos administrados por Hive Metastore. A diferencia de HDFS, no almacena directamente el dataset de Facebook, sino la información necesaria para describir las bases de datos y tablas de Hive. Su utilización permite conservar esa

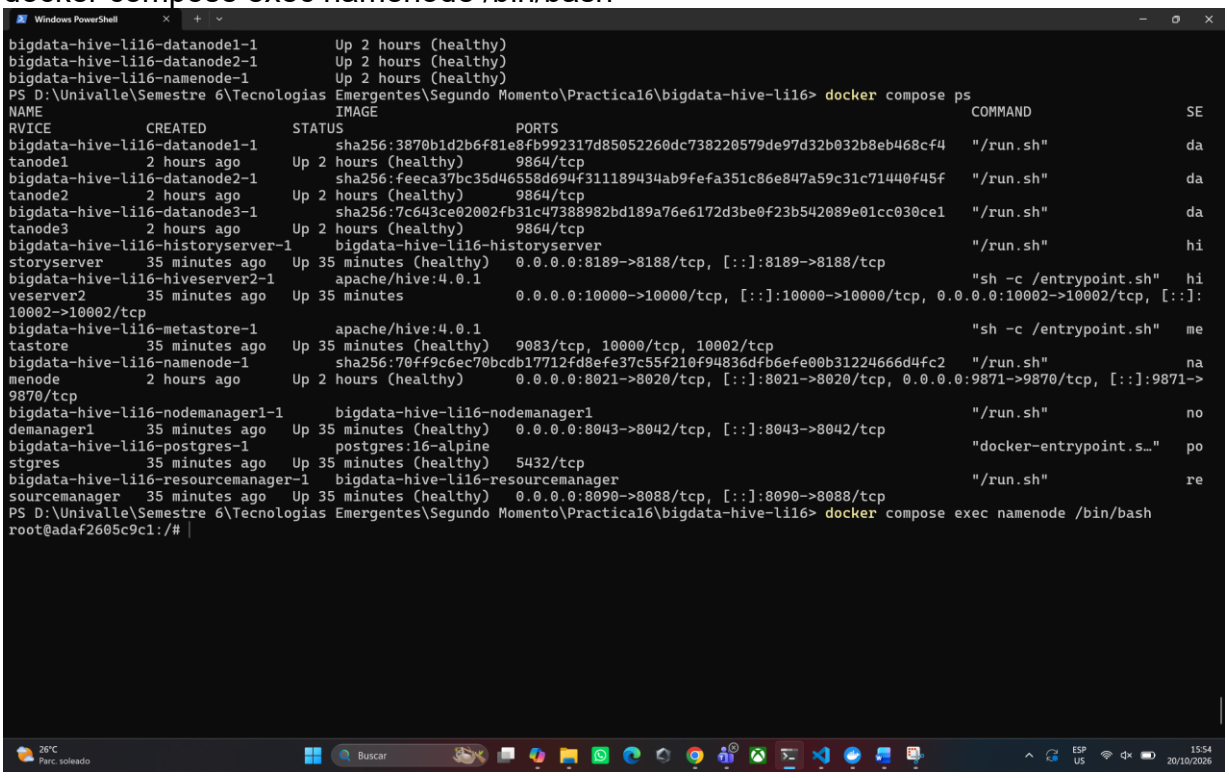
	información de manera persistente aunque los servicios de Hive se reinicien.
bigdata-hive-li16-namenode-1	El NameNode es el nodo principal del sistema HDFS y se encarga de administrar los metadatos de los archivos, directorios y ubicaciones de los bloques de datos. No almacena directamente el contenido de los archivos, sino que mantiene la información necesaria para localizarlos en los DataNodes. En esta práctica permitirá gestionar el archivo CSV que posteriormente cargaremos a HDFS.
bigdata-hive-li16-datanode1-1	El DataNode 1 es uno de los nodos encargados de almacenar físicamente los bloques de archivos dentro de HDFS. Trabaja bajo la coordinación del NameNode y participa en las operaciones de lectura, escritura y replicación de datos. En nuestra configuración se utiliza un factor de replicación de 2, por lo que este nodo puede almacenar bloques originales o réplicas junto con los otros DataNodes.
bigdata-hive-li16-datanode2-1	El DataNode 2 cumple la función de almacenar bloques de información y colaborar con los demás nodos del sistema distribuido. Su participación es importante porque HDFS distribuye los datos entre diferentes nodos y permite mantener copias para proteger la información. En esta práctica forma parte de la infraestructura que almacenará el dataset de Facebook y permitirá su consulta mediante Hive.
bigdata-hive-li16-datanode3-1	El DataNode 3 proporciona un nodo adicional de almacenamiento dentro del clúster HDFS. Su presencia permite distribuir los bloques y mantener copias en distintos nodos según el factor de replicación configurado. Comprendo que utilizar tres DataNodes con replicación 2 ayuda a mantener la disponibilidad de los datos cuando uno de los nodos presenta una falla, siempre que las réplicas necesarias estén disponibles.
bigdata-hive-li16-resourcemanager-1	El ResourceManager es un componente de Hadoop YARN encargado de administrar los recursos disponibles en el clúster y asignarlos a las aplicaciones que necesitan ejecutarse. Su función es coordinar el uso de memoria y capacidad de procesamiento. En nuestra infraestructura puede participar cuando las consultas de Hive requieren ejecutar trabajos distribuidos a través de Hadoop.
bigdata-hive-li16-nodemanager1-1	El NodeManager trabaja junto con el ResourceManager y se encarga de administrar los recursos y contenedores de procesamiento asignados a su nodo. También supervisa el consumo de recursos y comunica su estado al ResourceManager. Comprendo que ambos servicios se complementan para permitir la ejecución y el seguimiento de tareas distribuidas dentro de Hadoop.
bigdata-hive-li16-historyserver-1	El HistoryServer permite consultar información histórica sobre los trabajos MapReduce que finalizaron su ejecución, incluyendo sus resultados, tiempos y posibles errores. A diferencia del ResourceManager, su función principal no es asignar recursos, sino facilitar la revisión de trabajos anteriores. Este servicio resulta útil

	para analizar la ejecución de procesos de Hadoop y detectar problemas cuando se realizan tareas de procesamiento distribuido.



6. Acceder al shell del contenedor namenode con el comando: docker exec -it **docker-hive_namenode_1** /bin/bash

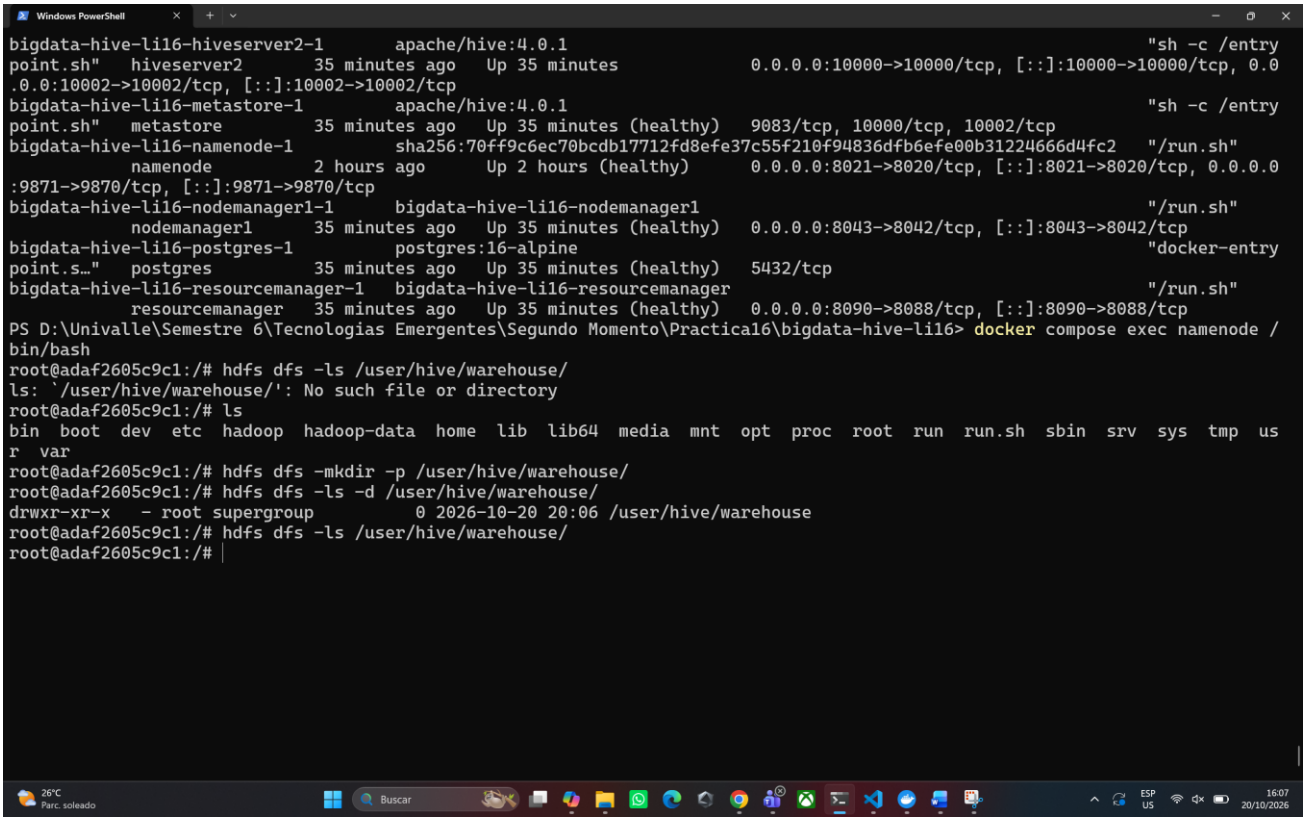
docker compose exec namenode /bin/bash



Se accedió a la terminal del contenedor NameNode mediante el comando ‘docker compose exec namenode /bin/bash’, adaptado a la arquitectura utilizada en esta práctica. Este acceso permite ejecutar comandos de Hadoop directamente desde el contenedor para administrar y consultar los archivos almacenados en HDFS.

7. Una vez que haya ingresado **al shell del contenedor docker-hive_namenode_1**, realice las siguientes tareas:
    a. Listar el HDFS con el comando: hdfs dfs -ls /user/hive/warehouse/

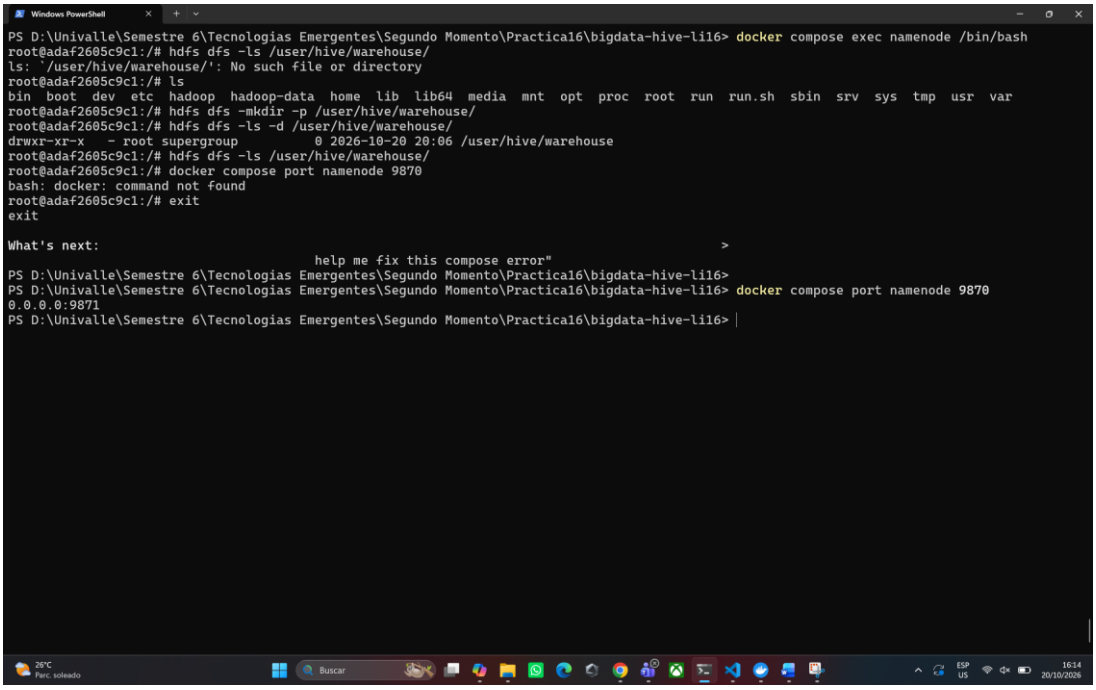
[Describa en un párrafo qué observa]

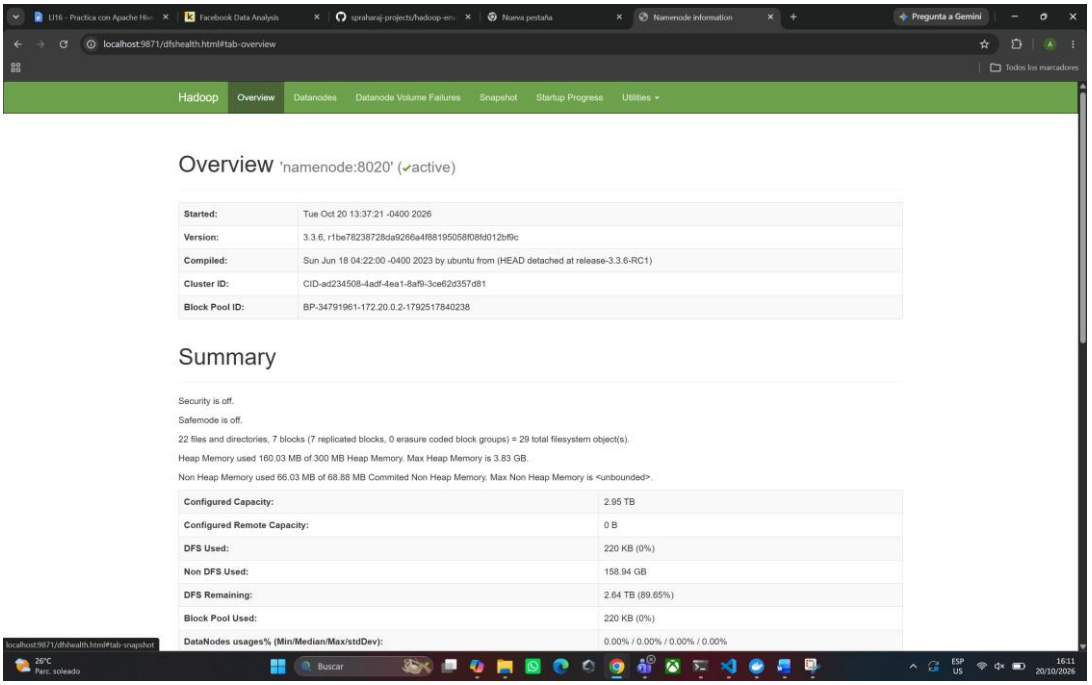


Al ejecutar el comando hdfs dfs -ls /user/hive/warehouse/, se observó que inicialmente el directorio no existía en HDFS. Por este motivo, fue necesario crearlo mediante el comando hdfs dfs -mkdir -p /user/hive/warehouse/. Posteriormente, se verificó su existencia y se volvió a listar su contenido. El comando se ejecutó correctamente, pero no mostró archivos porque todavía no se ha cargado el dataset de Facebook. Esto confirma que el directorio está preparado para almacenar los datos que utilizaremos en los siguientes puntos de la práctica.

8. Acceda a la interfaz web del **docker-hive_namenode_1**, valide cuál es su IP del host y a qué puerto está vinculado entre el Docker y el host.
    a. Como recordatorio, si ve algo asi 0.0.0.0:**1234** -> **4321 {Rojo es el puerto de cara al host, Verde es el puerto de cara al docker}.** Despliegue la interfaz web en el IP del host y puerto para el host.

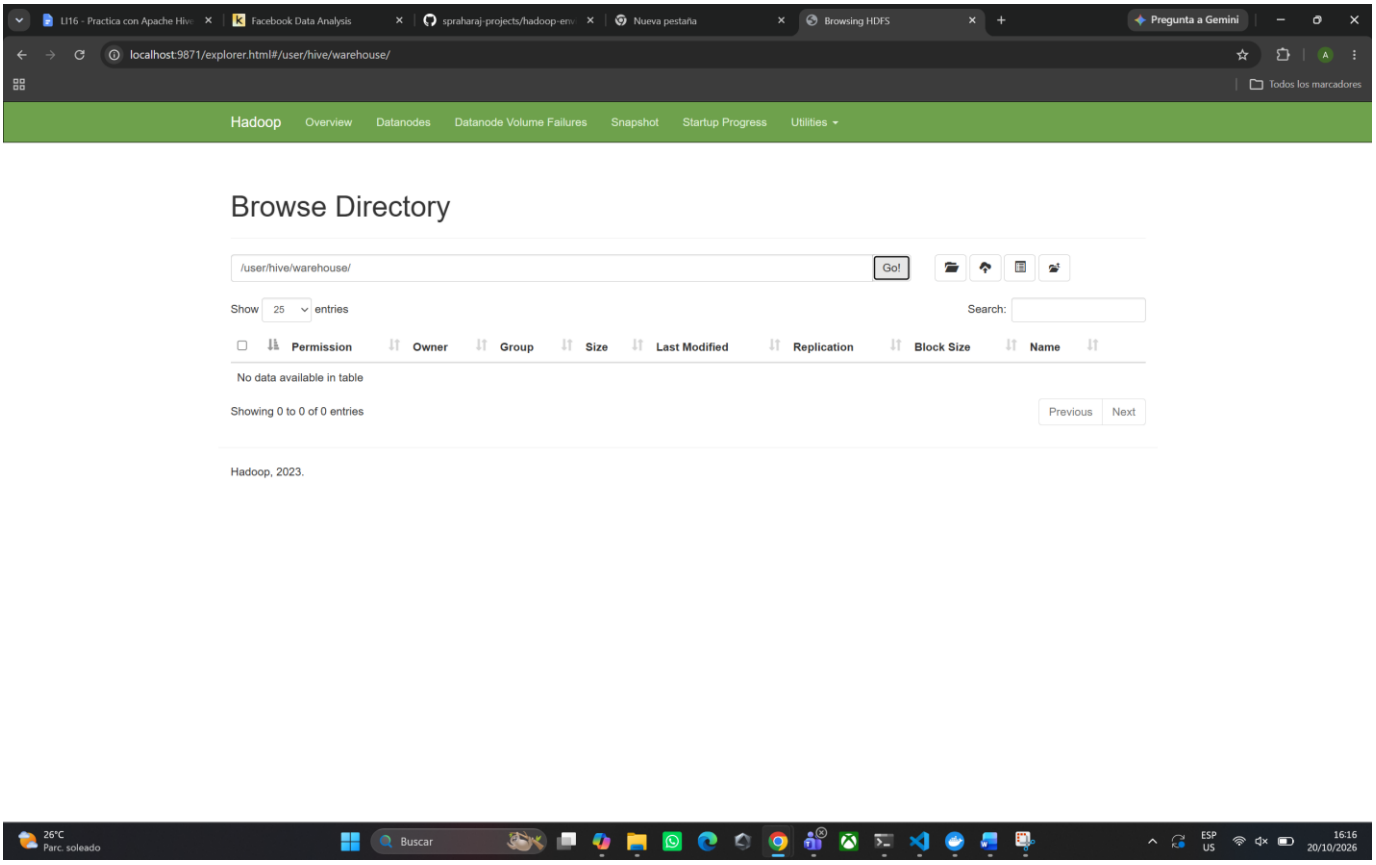
    docker compose port namenode 9870





Se accedió a la interfaz web del NameNode mediante la dirección http://localhost:9871, utilizando la dirección local 127.0.0.1 del equipo anfitrión. Se verificó que Docker vincula el puerto 9871 de Windows con el puerto interno 9870 del contenedor. La interfaz de Hadoop 3.3.6 se mostró correctamente y confirmó que el NameNode se encontraba activo, permitiendo consultar el estado y la información del sistema HDFS.

9. En la interfaz web podrá encontrar una opción Utilities -> Browse the file system, revise qué archivos se encuentran en la ruta /user/hive/warehouse/

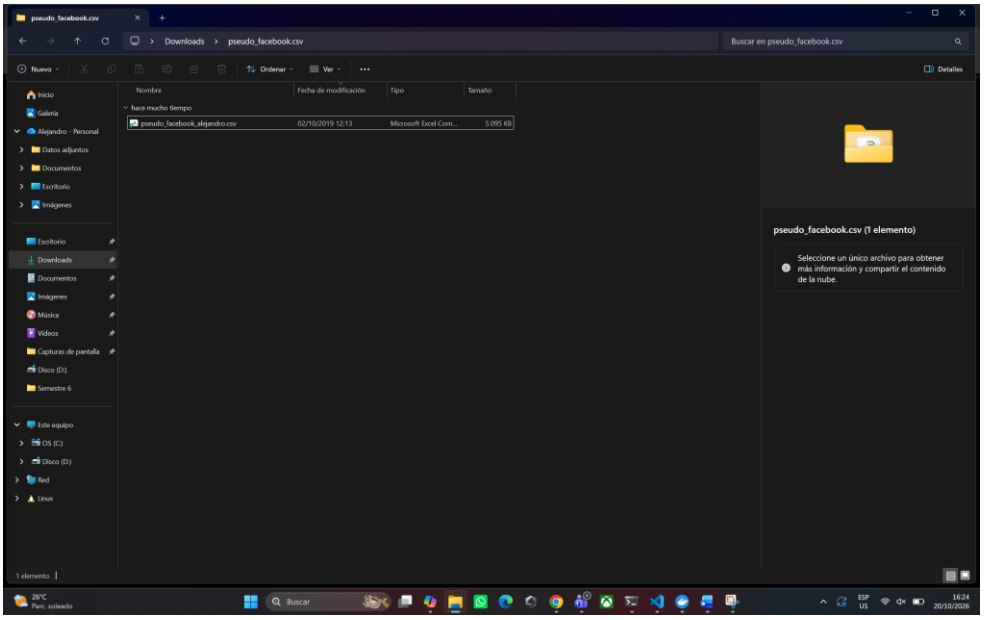


Se accedió a la interfaz web del NameNode mediante la opción Utilities -> Browse the file system y se consultó el directorio /user/hive/warehouse/. Se observó que el directorio existe, pero actualmente no contiene ningún archivo, por lo que la interfaz muestra cero registros. Esto se debe a que todavía no se ha cargado el dataset de Facebook en HDFS. Posteriormente, se utilizará esta misma ruta para verificar que el archivo CSV fue almacenado correctamente.

10. Descarga en la máquina host/anfitrión el siguiente dataset; debe tener claro dónde lo está descargando:

https://www.kaggle.com/code/youssefabdelghfar/facebook-data-analysis/input

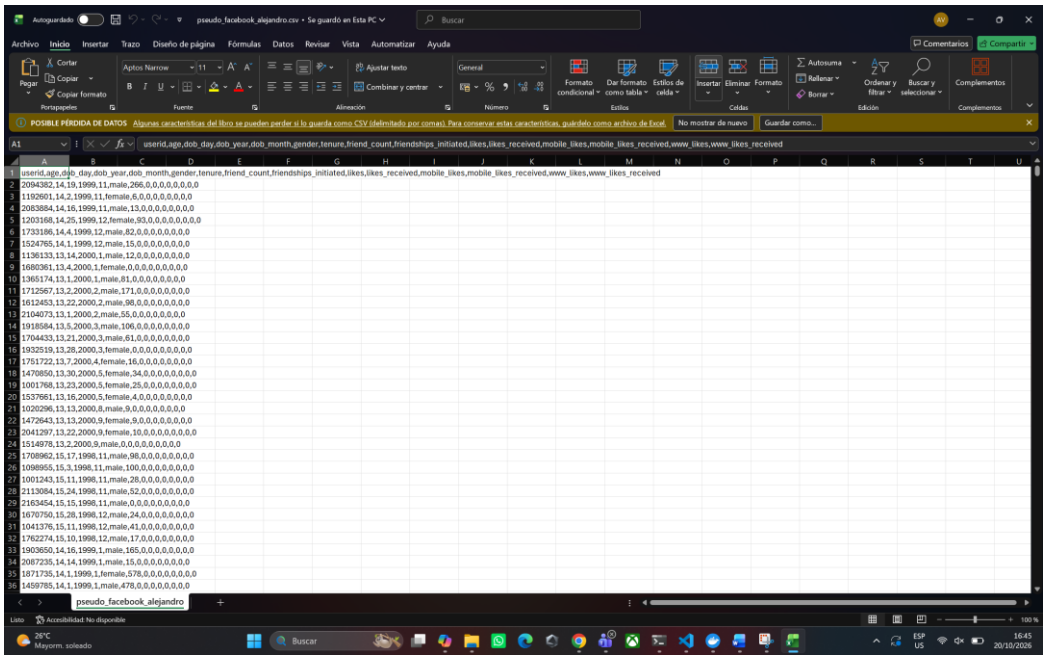
Descomprimir y cambiar el nombre a **pseudo_facebook_tunombre.csv (tomar en cuenta que debe estar tu nombre).**



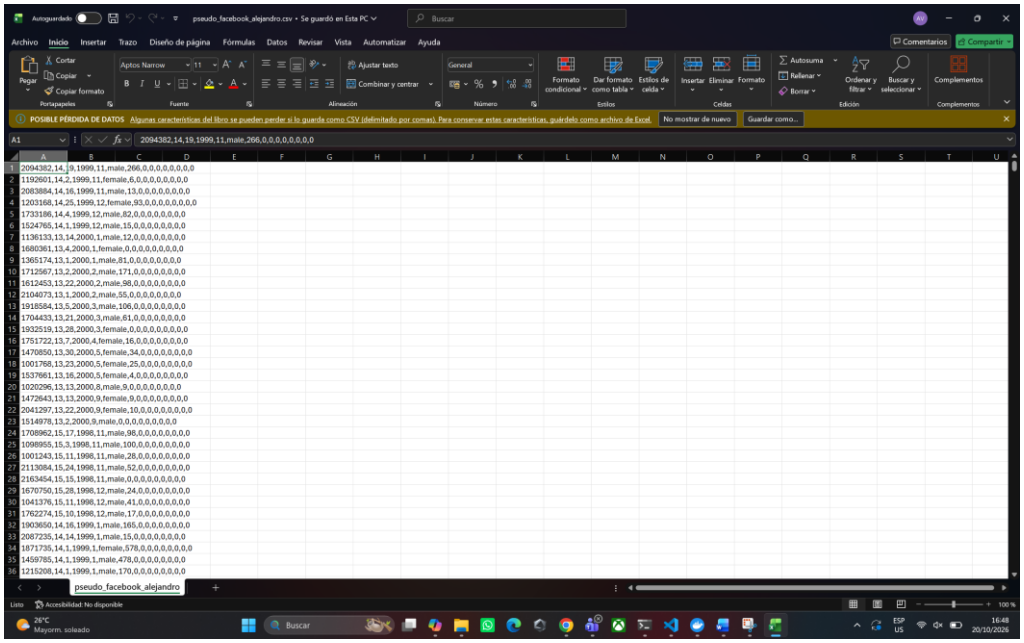
Se descargó el dataset ‘pseudo_facebook.csv’ desde la plataforma Kaggle. El archivo fue obtenido en formato ZIP y posteriormente descomprimido en la carpeta Descargas del equipo anfitrión. El archivo CSV resultante fue renombrado como ‘pseudo_facebook_alejandro.csv’, siguiendo la nomenclatura solicitada en la guía. El dataset contiene información de usuarios de Facebook que posteriormente será almacenada en HDFS y analizada mediante Apache Hive.

11. Quitar la primera línea o encabezado del archivo descargado para que no presente problema la carga de CSV a la tabla más adelante.

    Antes:



    Despues:

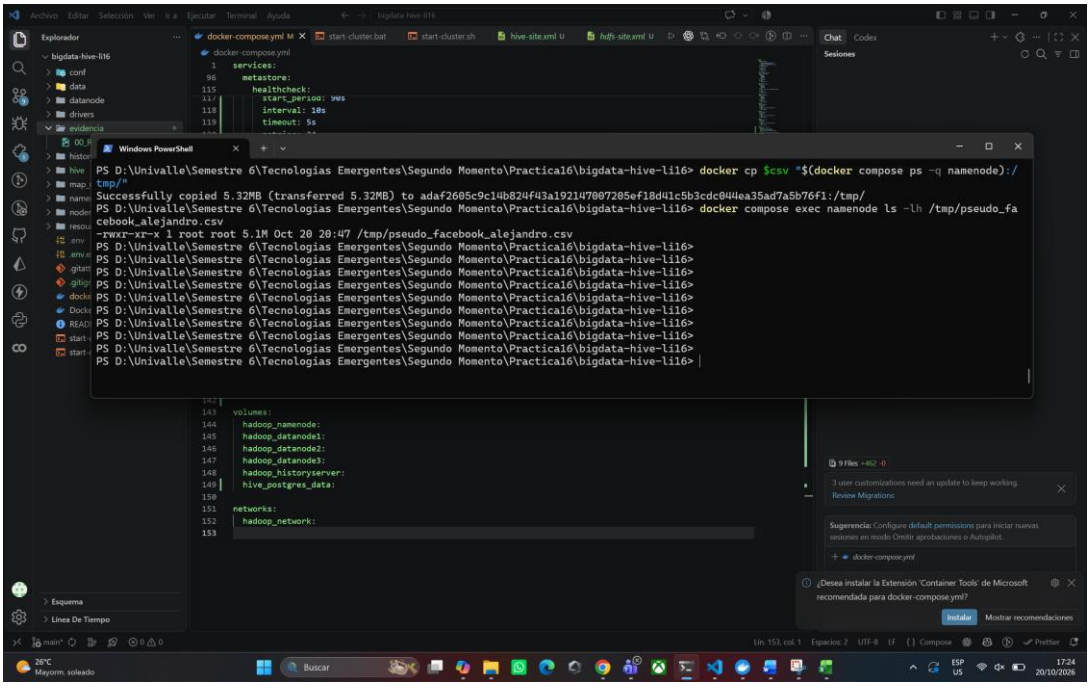


Se eliminó la primera línea del archivo ‘pseudo_facebook_alejandro.csv’, correspondiente a los nombres de las 15 columnas del dataset. Para evitar la pérdida de información, se creó previamente una copia de seguridad del archivo original. Posteriormente, se verificó que el CSV comienza directamente con los datos del primer usuario. Esta modificación permitirá cargar los registros a la tabla de Apache Hive sin interpretar el encabezado como un registro de datos.

12. Copiar el archivo **pseudo_facebook_tunombre.csv** del host a la ruta /tmp/ en el contenedor namenode con el siguiente comando:

docker cp **pseudo_facebook_tunombre.csv docker-hive_namenode_1**:/tmp/

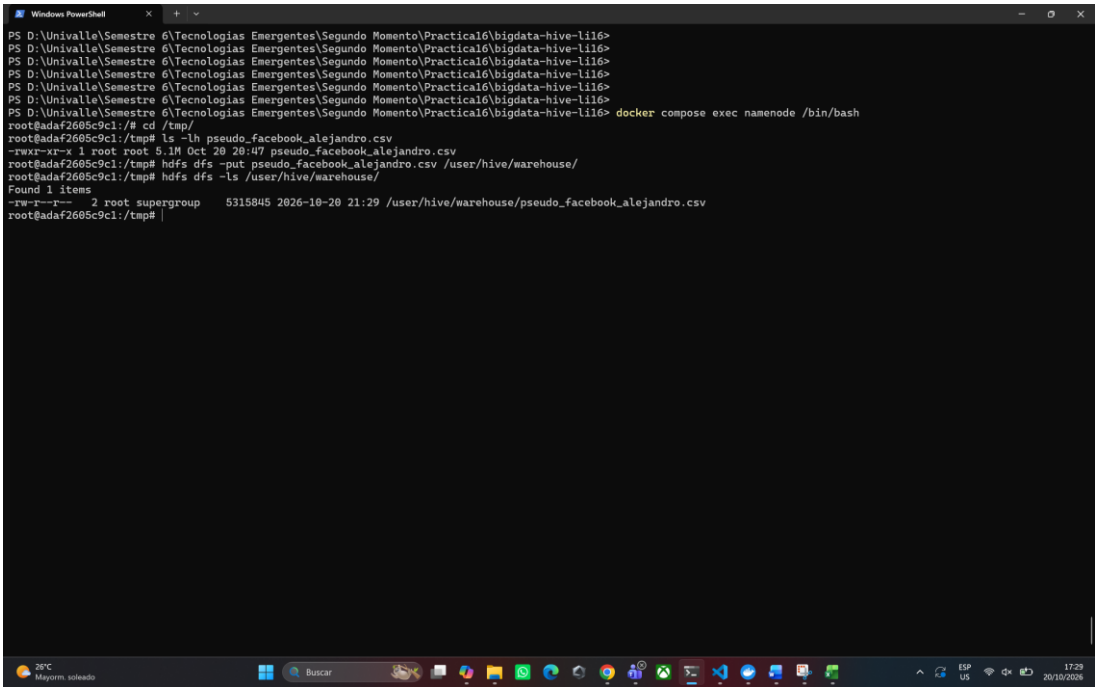
docker cp $csv "$(docker compose ps -q namenode):/tmp/"
docker compose exec namenode ls -lh /tmp/pseudo_facebook_alejandro.csv



Se copió el archivo ‘pseudo_facebook_alejandro.csv’ desde Windows hacia el directorio ‘/tmp/’ del contenedor NameNode mediante el comando ‘docker cp’. Posteriormente, se verificó que el archivo se encontraba en la ubicación indicada y tenía un tamaño aproximado de 5,1 MB. De esta manera, el dataset quedó preparado para subirlo a HDFS.

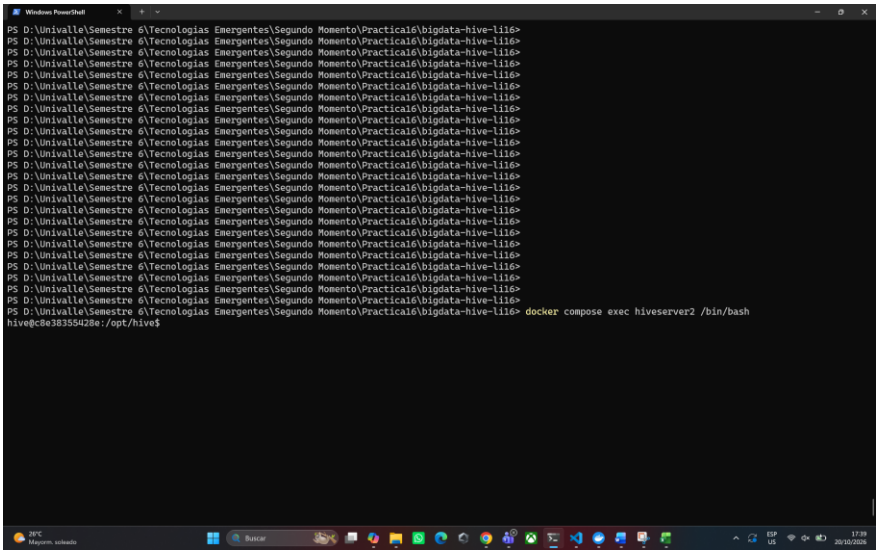
13. Ingresar al shell de namenode y realizar las siguientes tareas:
    a. Ingresar a la carpeta donde se copiaron los archivos del host con el comando: cd /tmp/
    b. Subir al HDFS el archivo a analizar denominado **pseudo_facebook_tunombre.csv** con el comando: hdfs dfs -put **pseudo_facebook_tunombre.csv** /user/hive/warehouse/
    c. Listar con el comando: hdfs dfs -ls /user/hive/warehouse/

    docker compose exec namenode /bin/bash
    cd /tmp/
    ls -lh pseudo_facebook_alejandro.csv
    hdfs dfs -put pseudo_facebook_alejandro.csv /user/hive/warehouse/
    hdfs dfs -ls /user/hive/warehouse/



    Se ingresó al contenedor NameNode y se accedió al directorio ‘/tmp/’, donde se encontraba el archivo ‘pseudo_facebook_alejandro.csv’. Posteriormente, se utilizó el comando ‘hdfs dfs -put’ para subirlo al directorio ‘/user/hive/warehouse/’ de HDFS. Finalmente, se verificó su almacenamiento mediante ‘hdfs dfs -ls’, obteniendo como resultado un archivo de 5.315.845 bytes. Esto confirma que el dataset está disponible en el sistema de archivos distribuido para su posterior utilización en Apache Hive.

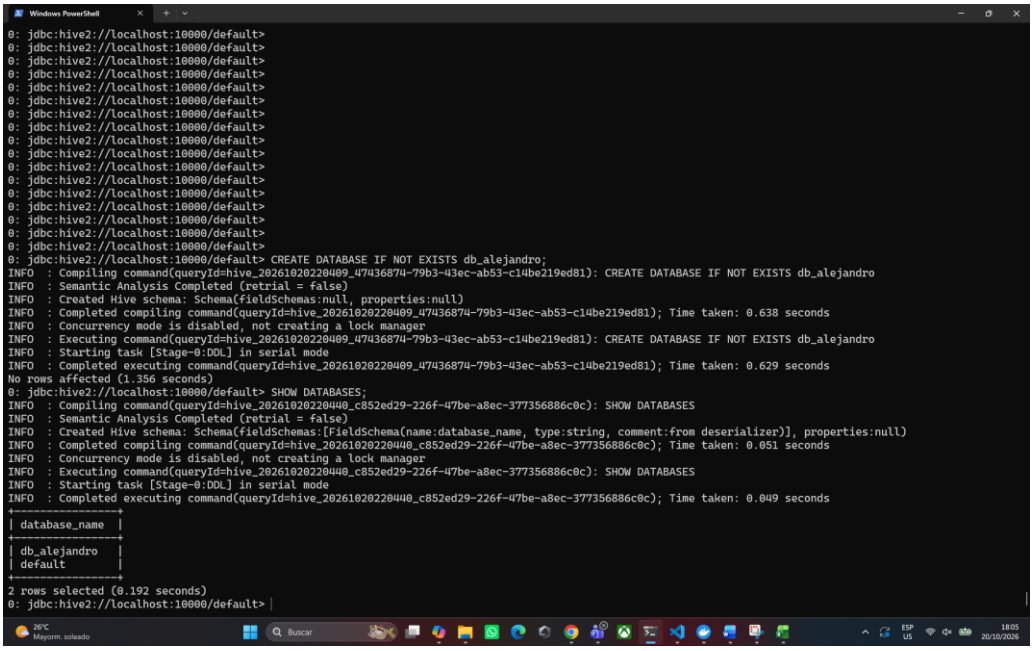
14. Acceder al shell de docker-hive_hive-server_1: docker exec -it **docker-hive_hive-server_1** /bin/bash



Se accedió al contenedor HiveServer2 mediante el comando docker compose exec hiveserver2 /bin/bash, adaptado a la arquitectura utilizada en nuestra práctica. Inicialmente, el contenedor se encontraba detenido, por lo que fue necesario recrearlo mediante Docker Compose. Posteriormente, se verificó que estaba en ejecución y se logró ingresar correctamente a su terminal Linux. Este acceso permitirá utilizar Apache Hive para crear bases de datos, definir tablas y realizar consultas SQL sobre el dataset almacenado en HDFS.

15. Acceder a hive desde el container **docker-hive_hive-server_1** con el comando: hive
    a. Crear una base de datos denominada **db_tunombre** con el comando:

create database **db_tunombre;**



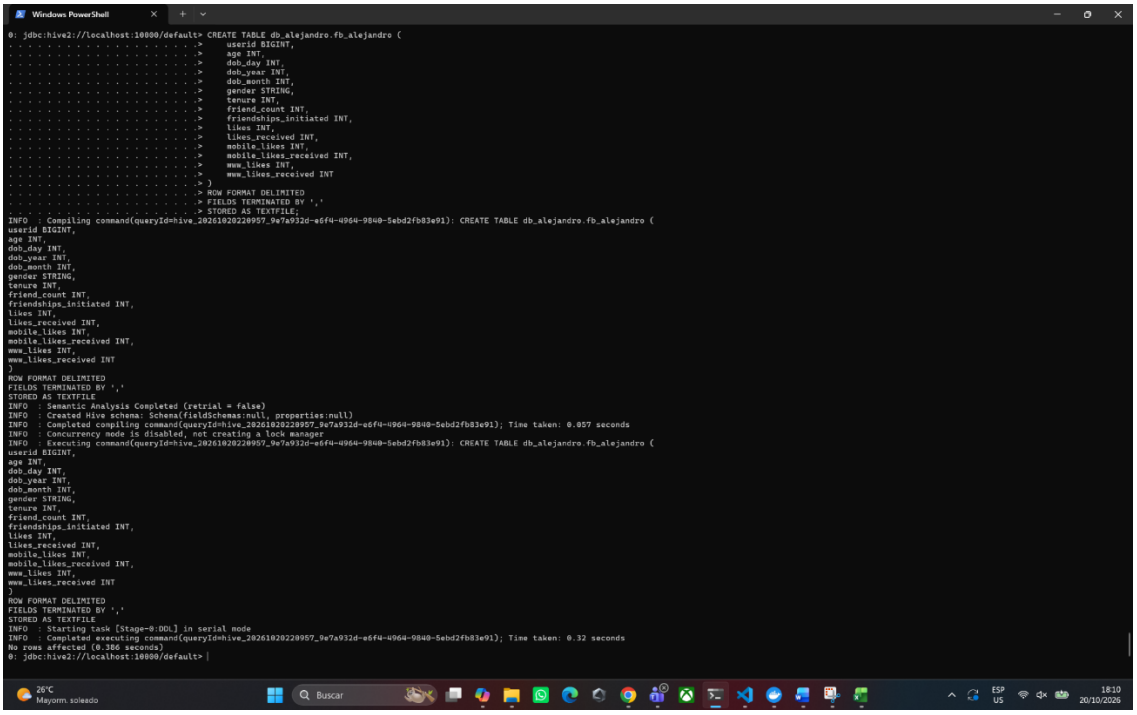
    b. Identifique qué columnas tiene su archivo **pseudo_facebook_tunombre.csv** y determine los tipos de datos que le corresponden:

Columna	Tipo de dato
userid	BIGINT
age	INT
dob_day	INT
dob_year	INT
dob_month	INT
gender	STRING
tenure	INT
friend_count	INT
friendships_initiated	INT
likes	INT
likes_received	INT
mobile_likes	INT
mobile_likes_received	INT
www_likes	INT
www_likes_received	INT

c. Crear una tabla denominada **fb_tunombre** tomando en cuenta los tipos de datos definidos anteriormente, con el comando:

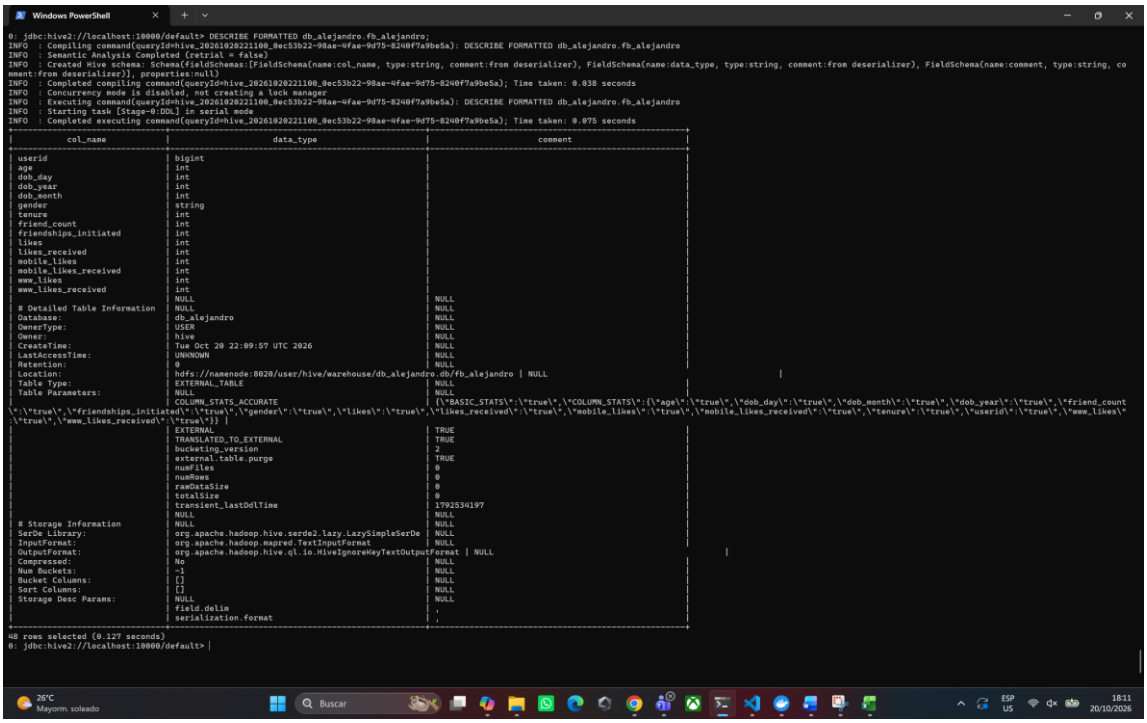
```
CREATE TABLE db_efrain.fb_tunombre(
id int, age int, day int, year int, month int,
gender string, tenure int, friends int, friends_init int,
likes int, likes_recd int, mlikes int, mlikes_rcd int,
wlikes int, wlikes_rcd int
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
```

STORED AS TEXTFILE;



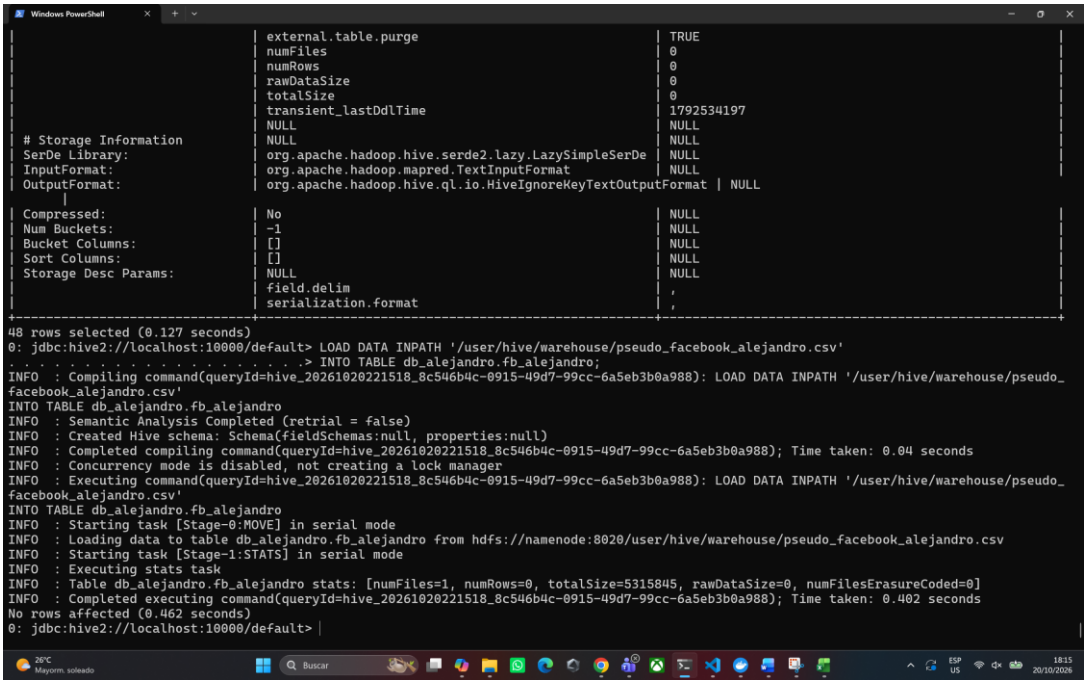
Se creó la tabla fb_alejandro dentro de la base de datos db_alejandro en Apache Hive. Se definieron las 15 columnas identificadas previamente, respetando su orden y tipo de dato. Además, se configuró el formato de almacenamiento como texto (TEXTFILE) y se estableció la coma como separador de campos para permitir la posterior carga del archivo CSV.

d. Revisa que el valor de location llegue hasta la tabla creada fb_tunombre con el comando: DESCRIBE FORMATTED db_tunombre.fb_tunombre;



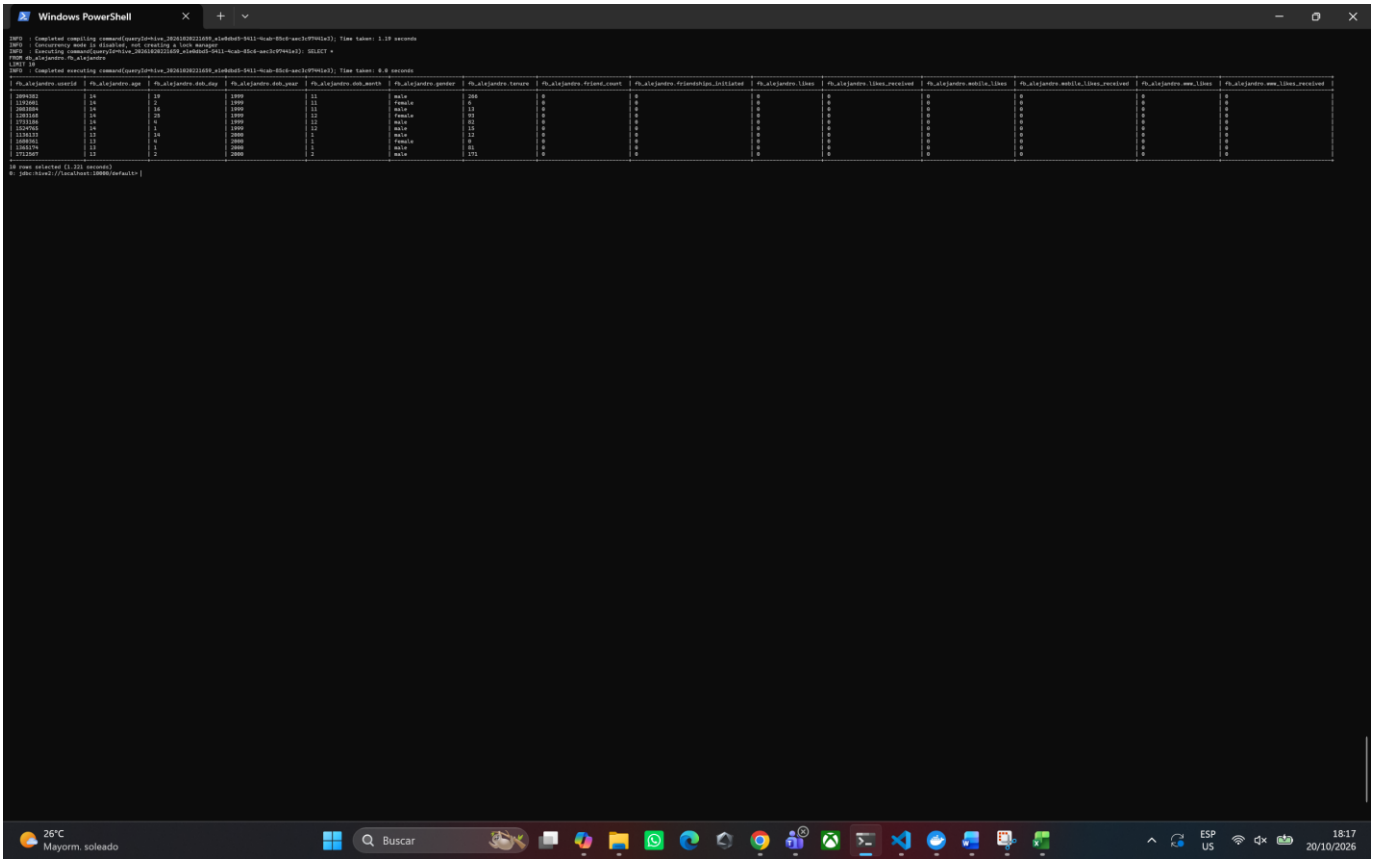
Se ejecutó la instrucción DESCRIBE FORMATTED sobre la tabla db_alejandro.fb_alejandro para comprobar su estructura. Esta consulta permite visualizar las columnas definidas, sus respectivos tipos de datos y las propiedades de almacenamiento configuradas en Apache Hive. De esta manera, se puede verificar que la tabla fue creada de acuerdo con la estructura del archivo CSV.

e. Cargar los datos del CSV a la tabla creada con el comando: LOAD DATA INPATH '/user/hive/warehouse/**pseudo_facebook_tunombre.csv**' INTO TABLE db_tunombre.fb_tunombre;



Se cargó el archivo pseudo_facebook_alejandro.csv, previamente almacenado en HDFS, en la tabla db_alejandro.fb_alejandro mediante la instrucción LOAD DATA INPATH. Apache Hive completó la operación correctamente, registrando un archivo de 5.315.845 bytes. De esta manera, los datos quedaron disponibles en la tabla para realizar consultas SQL.

f. Mostrar los primeros 10 registros con el comando: select * from fb_tunombre limit 10;

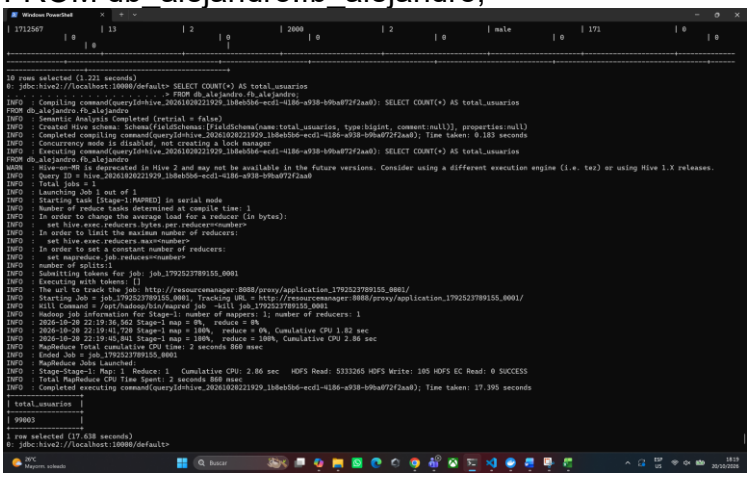


Se ejecutó una consulta SELECT * con la cláusula LIMIT 10 sobre la tabla db_alejandro.fb_alejandro. Como resultado, Apache Hive mostró los primeros 10 registros del dataset, distribuidos en las 15 columnas definidas. Esto permitió comprobar que los datos del archivo CSV fueron cargados correctamente y que pueden consultarse mediante instrucciones SQL.

16. Responder un párrafo en sus palabras después de cada evidencia, a los siguientes requerimientos en base a sus conocimientos de SQL:
    a. Determinar el número total de usuarios del dataset.

ConsultaSQL:

```
SELECT COUNT(*) AS total_usuarios
FROM db_alejandro.fb_alejandro;
```

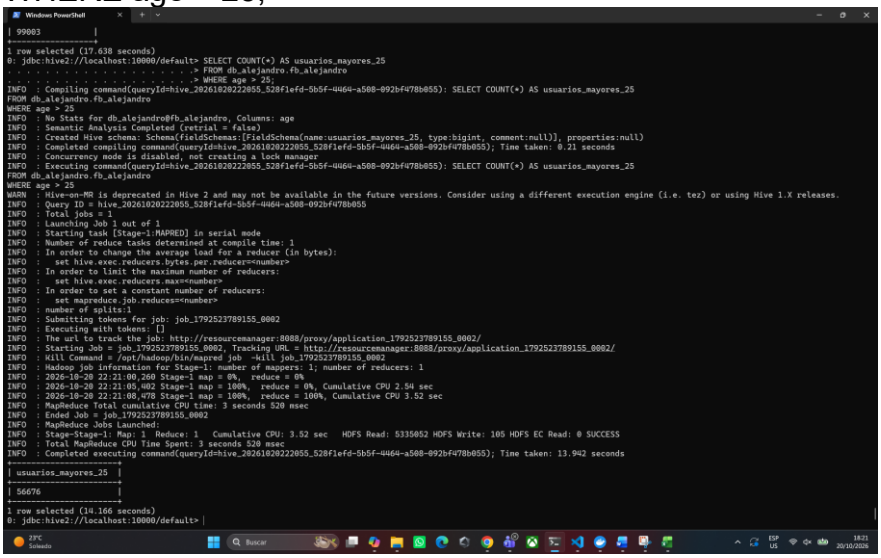


Se ejecutó la consulta SELECT COUNT(*) sobre la tabla db_alejandro.fb_alejandro para determinar la cantidad total de usuarios registrados. El resultado fue de 99.003 usuarios, cifra que coincide con los registros del archivo CSV después de eliminar su encabezado. Esto confirma que la carga de datos en Apache Hive se realizó correctamente y que todos los registros están disponibles para su análisis.

b. Determinar el número de usuarios de Facebook por encima de los 25 años.

ConsultaSQL:

```
SELECT COUNT(*) AS usuarios_mayores_25
FROM db_alejandro.fb_alejandro
WHERE age > 25;
```

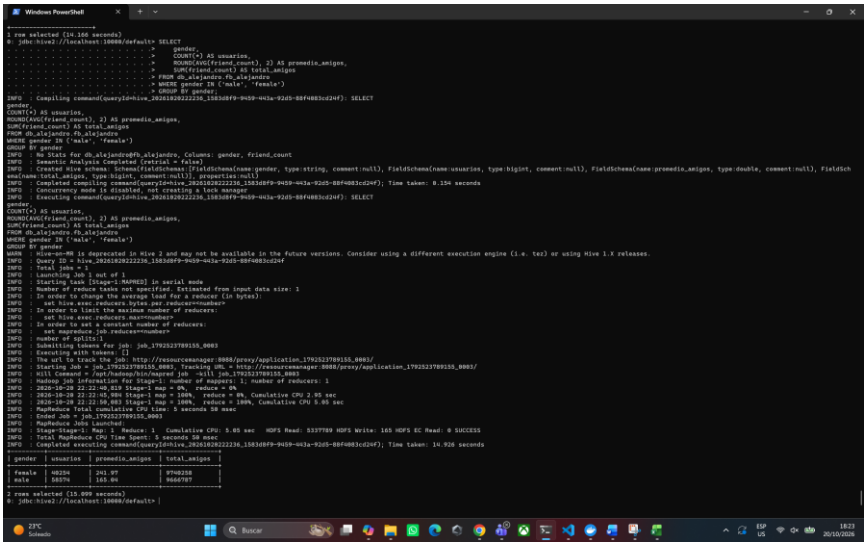


Se ejecutó una consulta SQL utilizando COUNT(*) y la condición WHERE age > 25 para determinar cuántos usuarios tienen más de 25 años. El resultado fue de 56.676 usuarios de un total de 99.003 registros. Esto demuestra que una parte mayoritaria de los usuarios del dataset supera los 25 años de edad.

c. Determinar si los usuarios masculinos o femeninos tienden a tener más amigos.

ConsultaSQL:

```
SELECT
    gender,
    COUNT(*) AS usuarios,
    ROUND(AVG(friend_count), 2) AS promedio_amigos,
    SUM(friend_count) AS total_amigos
FROM db_alejandro.fb_alejandro
WHERE gender IN ('male', 'female')
GROUP BY gender;
```

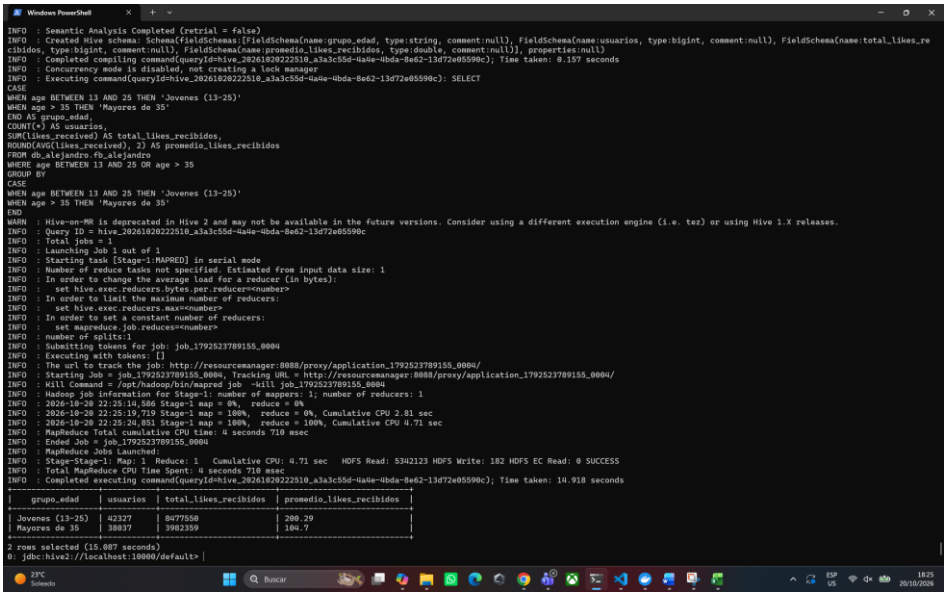


Se realizó una consulta SQL para comparar la cantidad de amigos entre hombres y mujeres, utilizando las funciones COUNT, AVG y SUM. Los resultados muestran que las mujeres tienen un promedio de 241,97 amigos, mientras que los hombres tienen 165,04. Aunque existen más usuarios masculinos (58.574) que femeninos (40.254), las mujeres presentan un promedio y un total acumulado de amigos superiores. Por lo tanto, se concluye que las mujeres tienen más amigos en promedio dentro del dataset analizado.

d. ¿Determinar cuántos ‘me gusta’ reciben los jóvenes (13-25 años) en Facebook en comparación con los usuarios mayores (mayores de 35 años)?

ConsultaSQL:

```
SELECT
   CASE
      WHEN age BETWEEN 13 AND 25 THEN 'Jovenes (13-25)'
      WHEN age > 35 THEN 'Mayores de 35'
   END AS grupo_edad,
   COUNT(*) AS usuarios,
   SUM(likes_received) AS total_likes_recibidos,
   ROUND(AVG(likes_received), 2) AS promedio_likes_recibidos
FROM db_alejandro.fb_alejandro
WHERE age BETWEEN 13 AND 25 OR age > 35
GROUP BY
   CASE
      WHEN age BETWEEN 13 AND 25 THEN 'Jovenes (13-25)'
      WHEN age > 35 THEN 'Mayores de 35'
   END;
```

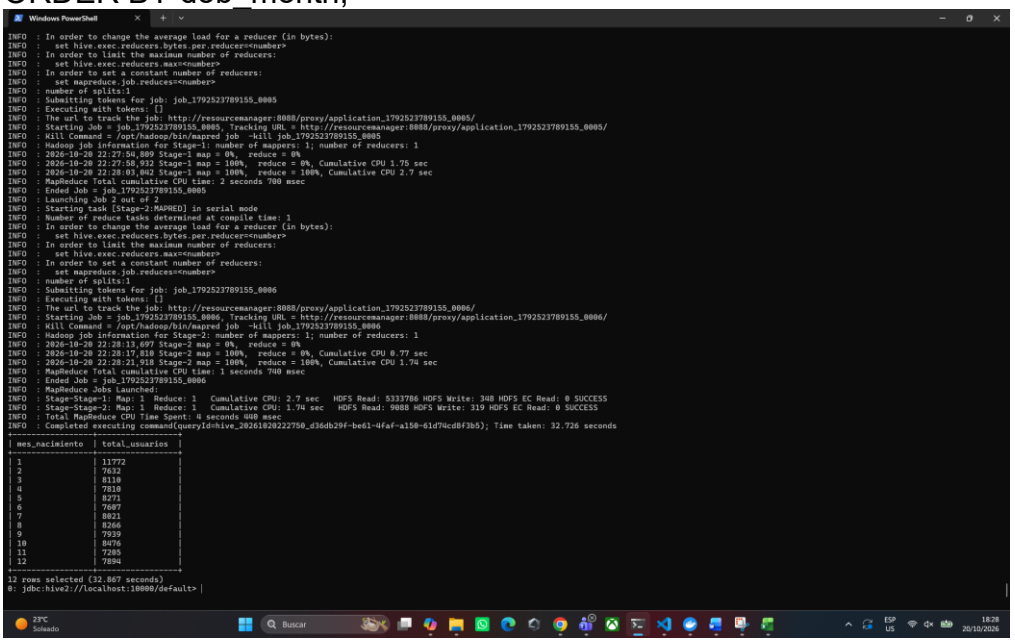


Se realizó una consulta SQL para comparar los likes recibidos por usuarios jóvenes de 13 a 25 años y usuarios mayores de 35 años. Los resultados muestran que los jóvenes tienen un promedio de 200,29 likes recibidos por usuario, mientras que los mayores de 35 años presentan un promedio de 104,70. Además, los jóvenes acumulan 8.477.550 likes recibidos, frente a 3.982.359 del grupo de mayor edad. Por lo tanto, se concluye que los usuarios jóvenes reciben más likes dentro del dataset analizado.

e. Encuentra el número de usuarios de Facebook para cada mes de cumpleaños.

ConsultaSQL:

```
SELECT
    dob_month AS mes_nacimiento,
    COUNT(*) AS total_usuarios
FROM db_alejandro.fb_alejandro
GROUP BY dob_month
ORDER BY dob_month;
```

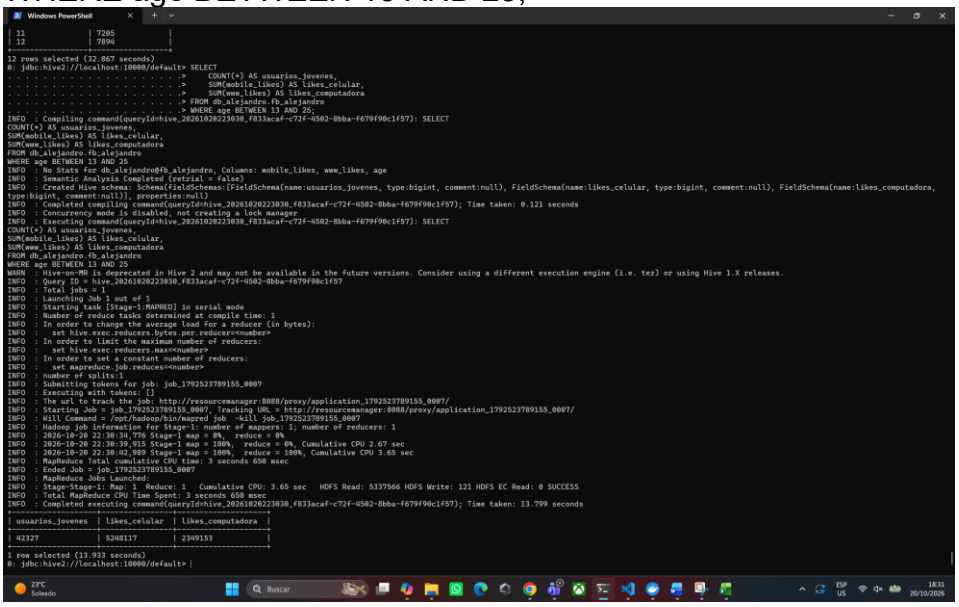


Se ejecutó una consulta SQL utilizando COUNT(*) y GROUP BY dob_month para determinar la cantidad de usuarios según su mes de nacimiento. Los resultados muestran que enero es el mes con mayor cantidad de usuarios registrados, con 11.772 personas, mientras que noviembre presenta la menor cantidad, con 7.205. Esto permite observar que la distribución de nacimientos varía entre los diferentes meses del año.

f. ¿Los jóvenes (13-25 años) usan más el celular o la computadora para usar Facebook?

ConsultaSQL:

```
SELECT
    COUNT(*) AS usuarios_jovenes,
    SUM(mobile_likes) AS likes_celular,
    SUM(www_likes) AS likes_computadora
FROM db_alejandro.fb_alejandro
WHERE age BETWEEN 13 AND 25;
```

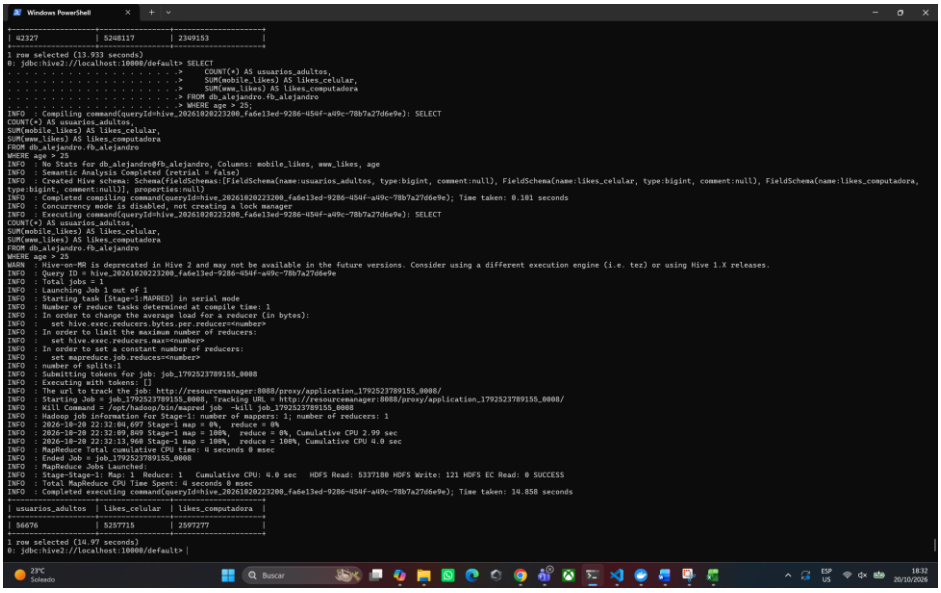


Se realizó una consulta SQL para comparar la actividad de los usuarios jóvenes de 13 a 25 años desde dispositivos móviles y desde la versión web de Facebook. Los resultados muestran que los 42.327 jóvenes realizaron 5.248.117 likes desde dispositivos móviles y 2.349.153 desde la versión web. Por lo tanto, se observa una mayor actividad mediante likes desde celulares. Sin embargo, esta comparación refleja los likes registrados y no necesariamente el tiempo total de uso de cada dispositivo.

g. ¿Los adultos (mayores de 25 años) usan más el celular o la computadora para usar Facebook?

ConsultaSQL:

```
SELECT
    COUNT(*) AS usuarios_adultos,
    SUM(mobile_likes) AS likes_celular,
    SUM(www_likes) AS likes_computadora
FROM db_alejandro.fb_alejandro
WHERE age > 25;
```



Se ejecutó una consulta SQL para comparar la actividad de los usuarios mayores de 25 años desde dispositivos móviles y desde la versión web de Facebook. Los resultados muestran que los 56.676 adultos realizaron 5.257.715 likes desde dispositivos móviles y 2.597.277 desde la versión web. Por lo tanto, se concluye que los adultos presentan una mayor actividad mediante likes desde celulares. Sin embargo, estos resultados representan la cantidad de likes realizados y no necesariamente el tiempo total de uso de cada dispositivo.

NOTA: Desarrollar un documento informe que presente una evidencia en imagen por cada punto requerido en el presente laboratorio; las condiciones de validez son:

- Debe apreciarse la hora de su sistema operativo.
- El contenido del documento debe ser mínimamente:
  - Portada que indique la materia, nombre del estudiante, número o código de práctica.
  - Debe mencionar el punto requerido (resaltado o pintado de amarillo) y la evidencia de respuesta que presenta al requerimiento; si considera necesario, puede incluir texto de aclaración.